

(SKILL Content Delivery Concepts)

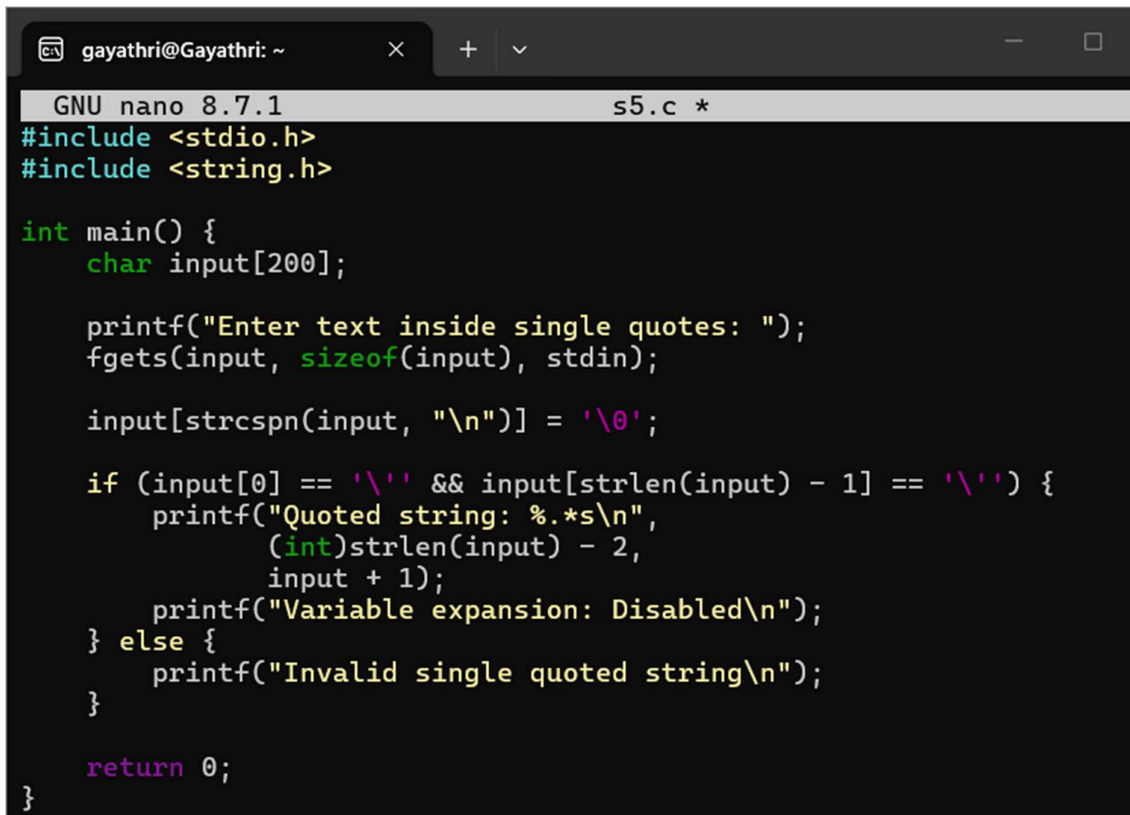
Name: G. Gayathri

Roll No: 2520030465

5.To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

5A) Code and output:



```
GNU nano 8.7.1 s5.c *
#include <stdio.h>
#include <string.h>

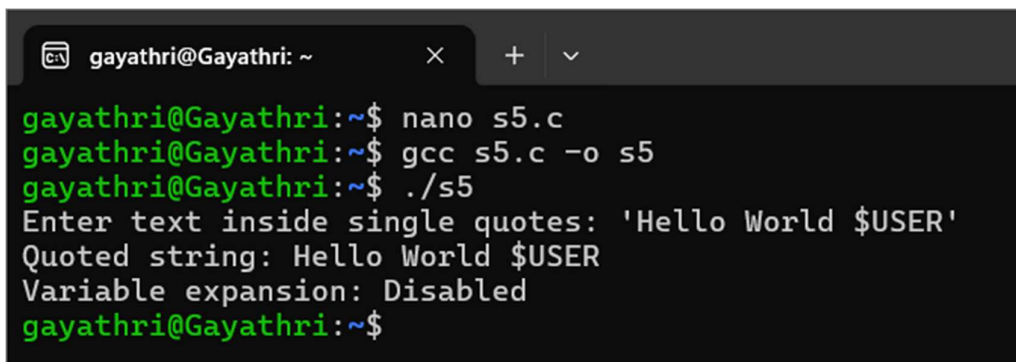
int main() {
    char input[200];

    printf("Enter text inside single quotes: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    if (input[0] == '\'' && input[strlen(input) - 1] == '\'' ) {
        printf("Quoted string: %.*s\n",
               (int)strlen(input) - 2,
               input + 1);
        printf("Variable expansion: Disabled\n");
    } else {
        printf("Invalid single quoted string\n");
    }

    return 0;
}
```



```
gayathri@Gayathri:~$ nano s5.c
gayathri@Gayathri:~$ gcc s5.c -o s5
gayathri@Gayathri:~$ ./s5
Enter text inside single quotes: 'Hello World $USER'
Quoted string: Hello World $USER
Variable expansion: Disabled
gayathri@Gayathri:~$
```

5B) Code and output:

```
gayathri@Gayathri: ~  
GNU nano 8.7.1 q5_double.c *  
#include <stdlib.h>  
#include <string.h>  
  
int main() {  
    char input[200];  
    char *value;  
  
    printf("Enter text inside double quotes: ");  
    fgets(input, sizeof(input), stdin);  
  
    input[strcspn(input, "\n")] = '\0';  
  
    if (input[0] == '"' && input[strlen(input) - 1] == '"') {  
        char content[200];  
  
        strncpy(content, input + 1, strlen(input) - 2);  
        content[strlen(input) - 2] = '\0';  
  
        value = getenv("USER");  
  
        if (value != NULL) {  
            char *pos = strstr(content, "$USER");  
  
            if (pos != NULL) {  
                char result[200];  
  
                int prefix = pos - content;  
  
                strncpy(result, content, prefix);  
                result[prefix] = '\0';  
  
                strcat(result, value);  
                strcat(result, pos + 5);  
  
                printf("Expanded string: %s\n", result);  
            } else {  
                printf("Expanded string: %s\n", content);  
            }  
        }  
    } else {  
        printf("Invalid double quoted string\n");  
    }  
  
    return 0;  
}
```

```
gayathri@Gayathri:~$ nano q5_double.c  
gayathri@Gayathri:~$ gcc q5_double.c -o q5_double  
gayathri@Gayathri:~$ ./q5_double  
Enter text inside double quotes: "Hello $USER"  
Expanded string: Hello gayathri  
gayathri@Gayathri:~$
```

6.To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

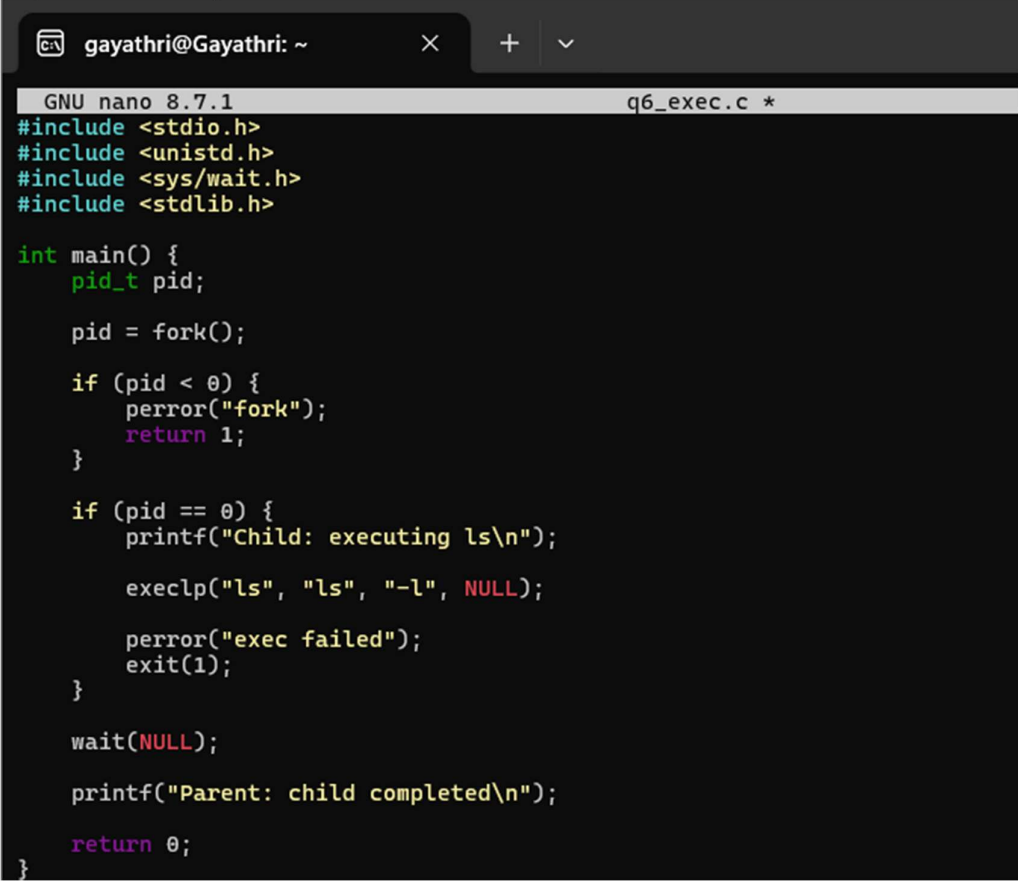
To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

Code:

6A) Code and output:

```
gayathri@Gayathri: ~  
GNU nano 8.7.1 q6_escape.c *  
#include <stdio.h>  
#include <string.h>  
  
int main() {  
    char input[200];  
    char output[200];  
    int i, j = 0;  
  
    printf("Enter text: ");  
    fgets(input, sizeof(input), stdin);  
  
    for (i = 0; input[i] != '\0'; i++) {  
        if (input[i] == '\\' && input[i + 1] != '\0') {  
            i++;  
            output[j++] = input[i];  
        } else {  
            output[j++] = input[i];  
        }  
    }  
  
    output[j] = '\0';  
  
    printf("Parsed output: %s", output);  
  
    return 0;  
}  
  
gayathri@Gayathri:~$ nano q6_escape.c  
gayathri@Gayathri:~$ gcc q6_escape.c -o q6_escape  
gayathri@Gayathri:~$ ./q6_escape  
Enter text: Hello\ World  
Parsed output: Hello World  
gayathri@Gayathri:~$
```

6B) Code and output:



```
GNU nano 8.7.1 q6_exec.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Child: executing ls\n");

        execlp("ls", "ls", "-l", NULL);

        perror("exec failed");
        exit(1);
    }

    wait(NULL);

    printf("Parent: child completed\n");

    return 0;
}
```

```
Process Synchronization Using Waitpid
parent@kali:~$ gcc pf_exe.c -o pf_exe
parent@kali:~$ ./pf_exe
Child scheduling is
Total 1200
-nsr--nsr- 1 gsyathr1 gsyathr1 16816 Sep 26 04:02 Address_Variable
-nsr--nsr- 1 gsyathr1 gsyathr1 158 Sep 26 04:07 Address_Variable.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16800 Aug 12 07:00 ChildMail
-nsr--nsr- 1 gsyathr1 gsyathr1 127 Aug 12 07:02 ChildMail.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16176 Aug 6 04:02 Copylibthwr
-nsr--nsr- 1 gsyathr1 gsyathr1 1592 Aug 6 04:02 Copylibthwr.c
-nsr--nsr- 1 gsyathr1 gsyathr1 15952 Aug 7 15:23 wlls
-nsr--nsr- 1 gsyathr1 gsyathr1 04 Aug 7 15:22 wlls.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16304 Aug 6 03:08 lvar
-nsr--nsr- 1 gsyathr1 gsyathr1 563 Aug 6 04:00 lvar.c
-nsr--nsr- 1 gsyathr1 gsyathr1 1684 Aug 12 06:07 MultipleChild
-nsr--nsr- 1 gsyathr1 gsyathr1 223 Aug 13 06:06 MultipleChild.c
-nsr--nsr- 1 gsyathr1 gsyathr1 006 Sep 22 05:21 no_psepect
-nsr--nsr- 2 gsyathr1 gsyathr1 4806 Jul 28 04:18 OSAB
-nsr--nsr- 1 gsyathr1 gsyathr1 103 Aug 13 04:11 PCC
-nsr--nsr- 1 gsyathr1 gsyathr1 16806 Aug 1 05:04 ParentChild
-nsr--nsr- 1 gsyathr1 gsyathr1 170 Aug 1 05:03 ParentChild.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16846 Aug 1 06:05 Pqld
-nsr--nsr- 1 gsyathr1 gsyathr1 221 Aug 1 06:06 Pqld.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16812 Aug 1 06:07 Pctwr
-nsr--nsr- 1 gsyathr1 gsyathr1 211 Aug 1 06:07 Pctwr.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16812 Sep 2 05:20 lqal
-nsr--nsr- 1 gsyathr1 gsyathr1 211 Sep 2 05:20 lqal.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16816 Sep 26 05:05 address_translation
-nsr--nsr- 1 gsyathr1 gsyathr1 176 Sep 26 05:05 address_translation.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16800 Aug 12 06:08 childofn
-nsr--nsr- 1 gsyathr1 gsyathr1 16176 Sep 3 03:52 client
-nsr--nsr- 1 gsyathr1 gsyathr1 821 Sep 3 03:52 client.c
-nsr--nsr- 1 gsyathr1 gsyathr1 8164 Aug 26 05:03 command_history.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16176 Aug 6 04:02 copyfile
-nsr--nsr- 1 gsyathr1 gsyathr1 134 Aug 6 04:06 copyfile.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16832 Sep 28 05:03 com
-nsr--nsr- 1 gsyathr1 gsyathr1 277 Sep 28 05:03 com.c
-nsr--nsr- 1 gsyathr1 gsyathr1 15952 Aug 13 04:00 scup
-nsr--nsr- 1 gsyathr1 gsyathr1 170 Aug 12 04:00 scup.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16806 Aug 27 13:07 fcl
-nsr--nsr- 1 gsyathr1 gsyathr1 216 Aug 27 13:07 fcl.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16846 Aug 27 04:02 fcl_write
-nsr--nsr- 1 gsyathr1 gsyathr1 205 Aug 27 04:00 fcl_write.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16136 Sep 9 06:02 fcl_read
-nsr--nsr- 1 gsyathr1 gsyathr1 240 Sep 9 06:02 fcl_read.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16846 Sep 9 06:00 fcl_write
-nsr--nsr- 1 gsyathr1 gsyathr1 220 Sep 9 06:00 fcl_write.c
-nsr--nsr- 1 gsyathr1 gsyathr1 210 Sep 9 06:00 fcl_write.c.new
-nsr--nsr- 1 gsyathr1 gsyathr1 16176 Sep 9 06:07 fcl_write
-nsr--nsr- 1 gsyathr1 gsyathr1 208 Sep 9 06:07 fcl_write.c
-nsr--nsr- 1 gsyathr1 gsyathr1 9 Jul 29 06:12 flls
-nsr--nsr- 1 gsyathr1 gsyathr1 6 Jul 29 06:25 flls.txt
-nsr--nsr- 1 gsyathr1 gsyathr1 16806 Aug 26 05:05 fmp
-nsr--nsr- 1 gsyathr1 gsyathr1 1803 Sep 28 05:25 fmp.c
-nsr--nsr- 1 gsyathr1 gsyathr1 15952 Aug 7 15:27 fmp
-nsr--nsr- 1 gsyathr1 gsyathr1 00 Aug 7 15:26 fmp.c
-nsr--nsr- 1 gsyathr1 gsyathr1 00 Aug 15:27 fmp.c++
-nsr--nsr- 1 gsyathr1 gsyathr1 4806 Aug 27 05:11 glibad-files
-nsr--nsr- 1 gsyathr1 gsyathr1 9 Jul 29 07:03 wlls.c
-nsr--nsr- 1 gsyathr1 gsyathr1 652 Aug 5 08:00 lvar.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16126 Sep 26 05:07 mailoc
-nsr--nsr- 1 gsyathr1 gsyathr1 16846 Sep 26 05:07 mailoc.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16160 Sep 26 04:00 memory
-nsr--nsr- 1 gsyathr1 gsyathr1 640 Sep 26 04:00 memory.c
-nsr--nsr- 1 gsyathr1 gsyathr1 9 Aug 27 04:02 myfcl
-nsr--nsr- 1 gsyathr1 gsyathr1 6 Jul 29 06:24 new_2170.new
-nsr--nsr- 1 gsyathr1 gsyathr1 6 Jul 29 06:24 new_2238.new
-nsr--nsr- 1 gsyathr1 gsyathr1 15960 Sep 26 05:11 null_pointer
-nsr--nsr- 1 gsyathr1 gsyathr1 16160 Aug 13 04:03 oqfwh
-nsr--nsr- 1 gsyathr1 gsyathr1 210 Sep 26 05:04 oqfwh.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16184 Sep 26 05:04 oqfwh.c
-nsr--nsr- 1 gsyathr1 gsyathr1 1620 Sep 26 05:04 oqfwh.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16800 Aug 13 04:12 pcc
-nsr--nsr- 1 gsyathr1 gsyathr1 103 Aug 13 04:15 pcc.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16306 Sep 2 05:07 plpw
-nsr--nsr- 1 gsyathr1 gsyathr1 843 Sep 2 05:07 plpw.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16806 Aug 12 05:12 practical3
-nsr--nsr- 1 gsyathr1 gsyathr1 482 Aug 13 05:12 practical3.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16126 Aug 13 05:15 practical3b
-nsr--nsr- 1 gsyathr1 gsyathr1 207 Aug 13 05:15 practical3b.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16216 Sep 2 07:00 practical5
-nsr--nsr- 1 gsyathr1 gsyathr1 560 Sep 2 07:00 practical5.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16072 Sep 2 06:18 practical6
-nsr--nsr- 1 gsyathr1 gsyathr1 671 Sep 2 06:17 practical6.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16160 Sep 20 06:21 practical7
-nsr--nsr- 1 gsyathr1 gsyathr1 200 Sep 20 06:21 practical7.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16072 Sep 2 06:00 practical8
-nsr--nsr- 1 gsyathr1 gsyathr1 483 Aug 13 05:18 practical8b
-nsr--nsr- 1 gsyathr1 gsyathr1 16800 Sep 29 06:30 practical8b
-nsr--nsr- 1 gsyathr1 gsyathr1 218 Sep 29 06:30 practical8b.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16806 Aug 27 16:02 priority
-nsr--nsr- 1 gsyathr1 gsyathr1 1119 Aug 27 16:02 priority.c
-nsr--nsr- 1 gsyathr1 gsyathr1 4806 Aug 12 06:00 process_id
-nsr--nsr- 1 gsyathr1 gsyathr1 16800 Aug 12 06:01 process_id
-nsr--nsr- 1 gsyathr1 gsyathr1 122 Aug 12 06:02 process_id.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16306 Aug 18 11:02 program1
-nsr--nsr- 1 gsyathr1 gsyathr1 660 Aug 18 10:05 program1.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16024 Aug 18 10:00 program2
-nsr--nsr- 1 gsyathr1 gsyathr1 1723 Aug 18 10:01 program2.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16024 Aug 18 10:01 program2
-nsr--nsr- 1 gsyathr1 gsyathr1 16302 Dec 6 03:21 qf_shdile
-nsr--nsr- 1 gsyathr1 gsyathr1 1670 Dec 6 03:21 qf_shdile.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16806 Dec 6 03:21 qf_shdile.c
-nsr--nsr- 1 gsyathr1 gsyathr1 485 Dec 6 03:13 qf_shdile.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16160 Dec 6 03:20 qf_shdile.c
-nsr--nsr- 1 gsyathr1 gsyathr1 421 Dec 6 03:24 qf_exe.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16184 Aug 27 16:06 read_write
-nsr--nsr- 1 gsyathr1 gsyathr1 1213 Aug 27 16:06 read_write.c
-nsr--nsr- 1 gsyathr1 gsyathr1 2197 Aug 27 05:08 r3.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16216 Aug 27 05:01 set
-nsr--nsr- 1 gsyathr1 gsyathr1 678 Aug 27 05:07 set.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16216 Dec 6 03:09 s5
-nsr--nsr- 1 gsyathr1 gsyathr1 212 Dec 6 03:08 s5.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16284 Sep 3 03:02 server
-nsr--nsr- 1 gsyathr1 gsyathr1 740 Sep 2 02:04 server.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16160 Sep 2 05:05 slghild
-nsr--nsr- 1 gsyathr1 gsyathr1 820 Sep 2 05:05 slghild.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16256 Sep 2 05:08 slghpw
-nsr--nsr- 1 gsyathr1 gsyathr1 551 Sep 2 05:08 slghpw.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16800 Sep 2 05:08 slghpw
-nsr--nsr- 1 gsyathr1 gsyathr1 449 Sep 2 05:08 slghpw.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16160 Sep 2 05:04 slghpw.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16112 Aug 27 15:02 slghpw.c
-nsr--nsr- 1 gsyathr1 gsyathr1 838 Aug 27 15:04 slghpw.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16176 Sep 26 05:02 sll15
-nsr--nsr- 1 gsyathr1 gsyathr1 505 Sep 26 05:02 sll15.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16176 Sep 26 05:04 sll16
-nsr--nsr- 1 gsyathr1 gsyathr1 605 Sep 26 05:04 sll16.c
-nsr--nsr- 1 gsyathr1 gsyathr1 740 Sep 3 03:02 sockel.c
-nsr--nsr- 1 gsyathr1 gsyathr1 4806 Sep 22 05:12 sock_management
-nsr--nsr- 1 gsyathr1 gsyathr1 16846 Sep 2 05:20 srtis.c
-nsr--nsr- 1 gsyathr1 gsyathr1 16840 Aug 13 04:26 srtis
-nsr--nsr- 1 gsyathr1 gsyathr1 554 Aug 13 04:26 srtis.c
Parent: child completed
parent@kali:~$
```

7.Process Synchronization and Child Process Monitoring Using waitpid().

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution.

7A) Code and output:

```
gayathri@Gayathri: ~  
GNU nano 8.7.1 q7_waitpid.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
#include <stdlib.h>  
  
int main() {  
    pid_t pid;  
    int status;  
  
    pid = fork();  
  
    if (pid == 0) {  
        printf("Child process running...\n");  
        sleep(2);  
        printf("Child process finished\n");  
        exit(0);  
    }  
  
    printf("Parent waiting for child...\n");  
  
    waitpid(pid, &status, 0);  
  
    if (WIFEXITED(status)) {  
        printf("Child exited with status %d\n",  
               WEXITSTATUS(status));  
    }  
  
    return 0;  
}  
  
gayathri@Gayathri:~$ nano q7_waitpid.c  
gayathri@Gayathri:~$ gcc q7_waitpid.c -o q7_waitpid  
gayathri@Gayathri:~$ ./q7_waitpid  
Parent waiting for child...  
Child process running...  
Child process finished  
Child exited with status 0  
gayathri@Gayathri:~$
```

7B) Code and Output:



```
GNU nano 8.7.1 q7_path.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main() {
    char command[100];
    char *path;
    char *dir;

    printf("Enter command: ");
    scanf("%s", command);

    path = getenv("PATH");

    if (path == NULL) {
        printf("PATH not found\n");
        return 1;
    }

    path = strdup(path);
    dir = strtok(path, ":");

    while (dir != NULL) {
        char fullpath[300];

        snprintf(fullpath, sizeof(fullpath),
                 "%s/%s", dir, command);

        if (access(fullpath, X_OK) == 0) {
            printf("Executable found: %s\n", fullpath);
            free(path);
            return 0;
        }

        dir = strtok(NULL, ":");
    }

    printf("Command not found\n");

    free(path);
    return 0;
}
```

```
gayathri@Gayathri:~$ nano q7_path.c
gayathri@Gayathri:~$ gcc q7_path.c -o q7_path
gayathri@Gayathri:~$ ./q7_path
Enter command: ls
Executable found: /usr/bin/ls
gayathri@Gayathri:~$
```

8.To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic.

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

8A) Code and Output:

```
gayathri@Gayathri: ~  
GNU nano 8.7.1 q8_variable.c *  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
  
int main() {  
    char variable[100];  
    char *value;  
  
    printf("Enter variable name: ");  
    scanf("%s", variable);  
  
    value = getenv(variable);  
  
    if (value != NULL)  
        printf("%s = %s\n", variable, value);  
    else  
        printf("Undefined variable\n");  
  
    return 0;  
}  
  
gayathri@Gayathri:~$ nano q8_variable.c  
gayathri@Gayathri:~$ gcc q8_variable.c -o q8_variable  
gayathri@Gayathri:~$ ./q8_variable  
Enter variable name: HOME  
HOME = /home/gayathri  
gayathri@Gayathri:~$ |
```

8B) Code and Output:

```
gayathri@Gayathri: ~  
GNU nano 8.7.1 q8_builtin.c *  
#include <stdio.h>  
#include <string.h>  
  
void execute_command(char *cmd) {  
    if (strcmp(cmd, "help") == 0)  
        printf("Available: help, hello, exit\n");  
  
    else if (strcmp(cmd, "hello") == 0)  
        printf("Hello from shell!\n");  
  
    else if (strcmp(cmd, "exit") == 0)  
        printf("Exiting...\n");  
  
    else  
        printf("Invalid command\n");  
}  
  
int main() {  
    char command[100];  
  
    printf("shell> ");  
    scanf("%s", command);  
  
    execute_command(command);  
  
    return 0;  
}  
  
gayathri@Gayathri:~$ nano q8_builtin.c  
gayathri@Gayathri:~$ gcc q8_builtin.c -o q8_builtin  
gayathri@Gayathri:~$ ./q8_builtin  
shell> help  
Available: help, hello, exit  
gayathri@Gayathri:~$
```

9.To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation.

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic.

9A) Code and Output:

```
gayathri@Gayathri: ~  
GNU nano 8.7.1 q9_cd.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <limits.h>  
  
int main() {  
    char path[PATH_MAX];  
  
    printf("Enter directory: ");  
    scanf("%s", path);  
  
    if (chdir(path) == 0) {  
        char cwd[PATH_MAX];  
  
        getcwd(cwd, sizeof(cwd));  
  
        printf("Directory changed successfully\n");  
        printf("Current directory: %s\n", cwd);  
    } else {  
        perror("chdir");  
    }  
  
    return 0;  
}  
  
gayathri@Gayathri:~$ nano q9_cd.c  
gayathri@Gayathri:~$ gcc q9_cd.c -o q9_cd  
gayathri@Gayathri:~$ ./q9_cd  
Enter directory: /tmp  
Directory changed successfully  
Current directory: /tmp  
gayathri@Gayathri:~$
```

9B) Code and Output:

```
gayathri@Gayathri: ~  
GNU nano 8.7.1 q9_dispatch.c *  
#include <stdio.h>  
#include <string.h>  
  
void execute(char *command) {  
    if (strcmp(command, "cd") == 0)  
        printf("CD command selected\n");  
  
    else if (strcmp(command, "pwd") == 0)  
        printf("PWD command selected\n");  
  
    else if (strcmp(command, "exit") == 0)  
        printf("Exit command selected\n");  
  
    else  
        printf("Unknown command\n");  
}  
  
int main() {  
    char command[50];  
  
    printf("shell> ");  
    scanf("%s", command);  
  
    execute(command);  
  
    return 0;  
}
```

```

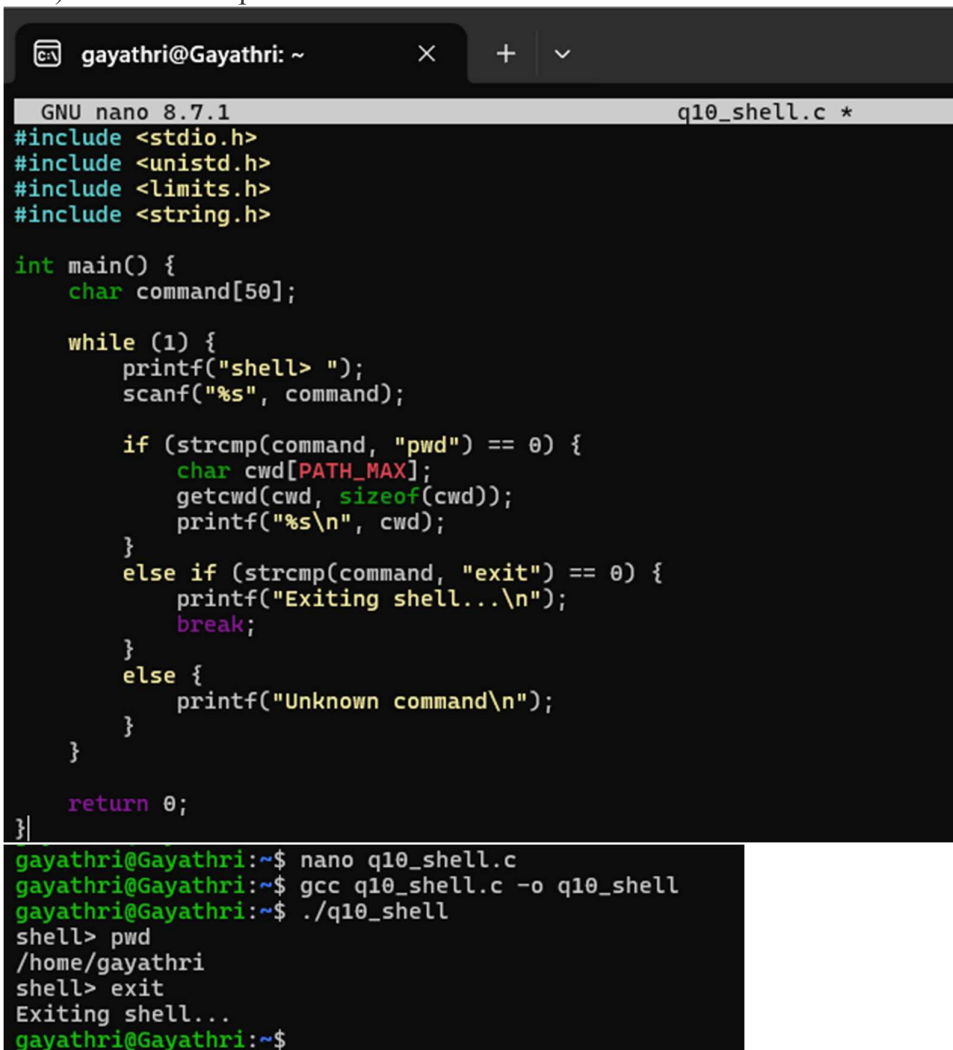
gayathri@Gayathri:~$ nano q9_dispatch.c
gayathri@Gayathri:~$ gcc q9_dispatch.c -o q9_dispatch
gayathri@Gayathri:~$ ./q9_dispatch
shell> pwd
PWD command selected
gayathri@Gayathri:~$

```

10. To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.

10A) Code and Output:



```

GNU nano 8.7.1 q10_shell.c *
#include <stdio.h>
#include <unistd.h>
#include <limits.h>
#include <string.h>

int main() {
    char command[50];

    while (1) {
        printf("shell> ");
        scanf("%s", command);

        if (strcmp(command, "pwd") == 0) {
            char cwd[PATH_MAX];
            getcwd(cwd, sizeof(cwd));
            printf("%s\n", cwd);
        }
        else if (strcmp(command, "exit") == 0) {
            printf("Exiting shell...\n");
            break;
        }
        else {
            printf("Unknown command\n");
        }
    }

    return 0;
}

```

```

gayathri@Gayathri:~$ nano q10_shell.c
gayathri@Gayathri:~$ gcc q10_shell.c -o q10_shell
gayathri@Gayathri:~$ ./q10_shell
shell> pwd
/home/gayathri
shell> exit
Exiting shell...
gayathri@Gayathri:~$

```

10B) Code and Output:

```
GNU nano 8.7.1 q10_export.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    char name[50];
    char value[100];

    printf("Enter variable name: ");
    scanf("%s", name);

    printf("Enter value: ");
    scanf("%s", value);

    if (setenv(name, value, 1) == 0)
        printf("Variable exported successfully\n");
    else
        perror("setenv");

    printf("%s=%s\n", name, getenv(name));

    return 0;
}

gayathri@Gayathri:~$ nano q10_export.c
gayathri@Gayathri:~$ gcc q10_export.c -o q10_export
gayathri@Gayathri:~$ ./q10_export
Enter variable name: CITY
Enter value: Hyderabad
Variable exported successfully
CITY=Hyderabad
gayathri@Gayathri:~$
```

11.To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

11A) Code and Output:

```
gayathri@Gayathri: ~  
GNU nano 8.7.1 q11_history.c *  
#include <stdio.h>  
#include <string.h>  
  
#define MAX_HISTORY 5  
  
int main() {  
    char history[MAX_HISTORY][100];  
    char command[100];  
    int count = 0;  
  
    printf("Enter commands (type exit):\n");  
  
    while (1) {  
        printf("shell> ");  
        fgets(command, sizeof(command), stdin);  
  
        command[strcspn(command, "\n")] = '\0';  
  
        if (strcmp(command, "exit") == 0)  
            break;  
  
        if (count < MAX_HISTORY) {  
            strcpy(history[count], command);  
            count++;  
        }  
    }  
  
    printf("\nCommand History:\n");  
  
    for (int i = 0; i < count; i++)  
        printf("%d: %s\n", i + 1, history[i]);  
  
    return 0;  
}
```

```
gayathri@Gayathri:~$ nano q11_history.c  
gayathri@Gayathri:~$ gcc q11_history.c -o q11_history  
gayathri@Gayathri:~$ ./q11_history  
Enter commands (type exit):  
shell> ls  
shell> pwd  
shell> date  
shell> exit  
  
Command History:  
1: ls  
2: pwd  
3: date  
gayathri@Gayathri:~$ |
```

11B) Code and Output:

```
gayathri@Gayathri: ~
GNU nano 8.7.1 q11_pipeline.c *
#include <stdio.h>
#include <string.h>

int main() {
    char input[200];
    char *command;

    printf("Enter pipeline: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    command = strtok(input, "|");

    printf("Pipeline commands:\n");

    while (command != NULL) {
        printf("-> %s\n", command);
        command = strtok(NULL, "|");
    }

    return 0;
}
```

```
gayathri@Gayathri:~$ nano q11_pipeline.c
gayathri@Gayathri:~$ gcc q11_pipeline.c -o q11_pipeline
gayathri@Gayathri:~$ ./q11_pipeline
Enter pipeline: ls
Pipeline commands:
-> ls
gayathri@Gayathri:~$ ./q11_pipeline
Enter pipeline: grep text | wc -l
Pipeline commands:
-> grep text
-> wc -l
gayathri@Gayathri:~$ ./q11_pipeline
Enter pipeline: ls | grep txt | wc -l
Pipeline commands:
-> ls
-> grep txt
-> wc -l
gayathri@Gayathri:~$
```

12.To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines.

12A) Code and Output:

```
gayathri@Gayathri: ~  
GNU nano 8.7.1 q12_pipe.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main() {  
    int fd[2];  
  
    pipe(fd);  
  
    if (fork() == 0) {  
        dup2(fd[1], STDOUT_FILENO);  
  
        close(fd[0]);  
        close(fd[1]);  
  
        execlp("ls", "ls", NULL);  
    }  
  
    if (fork() == 0) {  
        dup2(fd[0], STDIN_FILENO);  
  
        close(fd[0]);  
        close(fd[1]);  
  
        execlp("wc", "wc", "-l", NULL);  
    }  
  
    close(fd[0]);  
    close(fd[1]);  
  
    wait(NULL);  
    wait(NULL);  
  
    return 0;  
}  
|  
gayathri@Gayathri:~$ nano q12_pipe.c  
gayathri@Gayathri:~$ gcc q12_pipe.c -o q12_pipe  
gayathri@Gayathri:~$ ./q12_pipe  
155  
gayathri@Gayathri:~$
```

12B) Code and Output:

```
gayathri@Gayathri: ~  
GNU nano 8.7.1 q12_multipipe.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
int main() {  
    int p1[2], p2[2];  
    pipe(p1);  
    pipe(p2);  
    if (fork() == 0) {  
        dup2(p1[1], STDOUT_FILENO);  
        close(p1[0]);  
        close(p1[1]);  
        close(p2[0]);  
        close(p2[1]);  
        execlp("ls", "ls", NULL);  
    }  
    if (fork() == 0) {  
        dup2(p1[0], STDIN_FILENO);  
        dup2(p2[1], STDOUT_FILENO);  
        close(p1[0]);  
        close(p1[1]);  
        close(p2[0]);  
        close(p2[1]);  
        execlp("grep", "grep", "txt", NULL);  
    }  
    if (fork() == 0) {  
        dup2(p2[0], STDIN_FILENO);  
        close(p1[0]);  
        close(p1[1]);  
        close(p2[0]);  
        close(p2[1]);  
        execlp("wc", "wc", "-l", NULL);  
    }  
    close(p1[0]);  
    close(p1[1]);  
    close(p2[0]);  
    close(p2[1]);  
    wait(NULL);  
    wait(NULL);  
    wait(NULL);  
    return 0;  
}  
  
gayathri@Gayathri:~$ nano q12_multipipe.c  
gayathri@Gayathri:~$ gcc q12_multipipe.c -o q12_multipipe  
gayathri@Gayathri:~$ ./q12_multipipe  
1  
gayathri@Gayathri:~$
```

13.To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs.

To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection.

13A) Code and Output:

```
gayathri@Gayathri: ~  
GNU nano 8.7.1 q13_input.c *  
#include <stdio.h>  
#include <fcntl.h>  
#include <unistd.h>  
  
int main() {  
    int fd = open("input.txt", O_RDONLY);  
  
    if (fd < 0) {  
        perror("open");  
        return 1;  
    }  
  
    dup2(fd, STDIN_FILENO);  
    close(fd);  
  
    char buffer[500];  
    int n = read(STDIN_FILENO, buffer, sizeof(buffer) - 1);  
  
    buffer[n] = '\0';  
  
    printf("File contents:\n%s", buffer);  
  
    return 0;  
}  
  
gayathri@Gayathri:~$ echo "Hello OSSP" > input.txt  
gayathri@Gayathri:~$ nano q13_input.c  
gayathri@Gayathri:~$ gcc q13_input.c -o q13_input  
gayathri@Gayathri:~$ ./q13_input  
File contents:  
Hello OSSP  
gayathri@Gayathri:~$ |
```

13B) Code and Output:

```
gayathri@Gayathri: ~  
GNU nano 8.7.1 q13_output.c *  
#include <stdio.h>  
#include <fcntl.h>  
#include <unistd.h>  
  
int main() {  
    int fd = open("output.txt",  
                  O_WRONLY | O_CREAT | O_TRUNC,  
                  0644);  
  
    if (fd < 0) {  
        perror("open");  
        return 1;  
    }  
  
    dup2(fd, STDOUT_FILENO);  
    close(fd);  
  
    printf("Hello from redirected output!\n");  
  
    return 0;  
}|
```

```

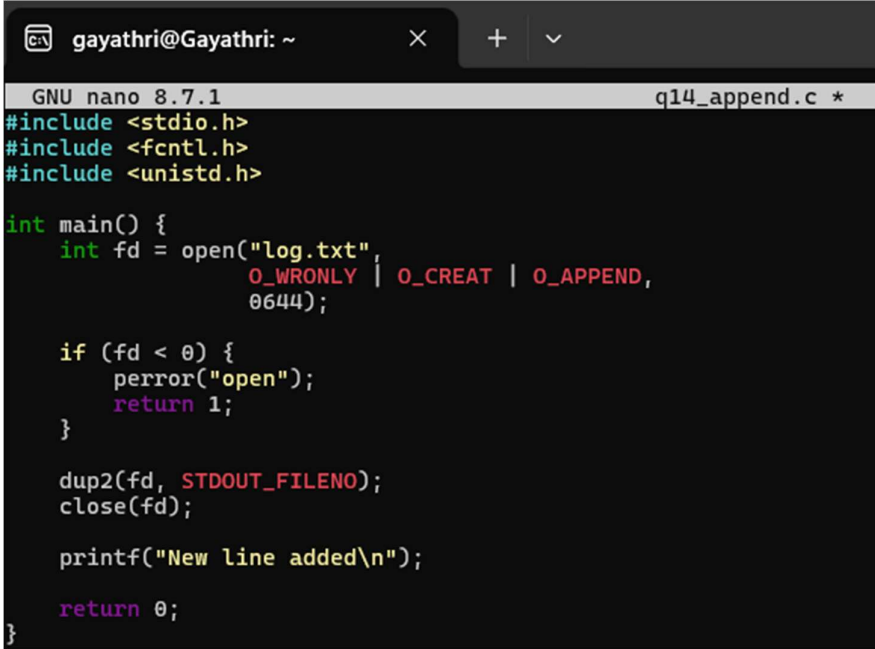
gayathri@Gayathri:~$ nano q13_output.c
gayathri@Gayathri:~$ gcc q13_output.c -o q13_output
gayathri@Gayathri:~$ ./q13_output
gayathri@Gayathri:~$ cat output.txt
Hello from redirected output!
gayathri@Gayathri:~$

```

14.To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

To Parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases.

14A) Code and Output:



```

GNU nano 8.7.1 q14_append.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("log.txt",
                  O_WRONLY | O_CREAT | O_APPEND,
                  0644);

    if (fd < 0) {
        perror("open");
        return 1;
    }

    dup2(fd, STDOUT_FILENO);
    close(fd);

    printf("New line added\n");

    return 0;
}

```

```

Hello from redirected output!
gayathri@Gayathri:~$ nano q14_append.c
gayathri@Gayathri:~$ gcc q14_append.c -o q14_append
gayathri@Gayathri:~$ ./q14_append
gayathri@Gayathri:~$ ./q14_append
gayathri@Gayathri:~$ cat log.txt
New line added
New line added
gayathri@Gayathri:~$

```

14B) Code and Output:

```
GNU nano 8.7.1 q14_stderr.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("error.txt",
                  O_WRONLY | O_CREAT | O_TRUNC,
                  0644);

    if (fd < 0)
        return 1;

    dup2(fd, STDERR_FILENO);
    close(fd);

    fprintf(stderr, "This is an error message\n");

    return 0;
}

gayathri@Gayathri:~$ nano q14_stderr.c
gayathri@Gayathri:~$ gcc q14_stderr.c -o q14_stderr
gayathri@Gayathri:~$ ./q14_stderr
gayathri@Gayathri:~$ cat error.txt
This is an error message
gayathri@Gayathri:~$
```

15.To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios.

15A) Code and Output:

```
GNU nano 8.7.1 q15_combined.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("combined.txt",
                  O_WRONLY | O_CREAT | O_TRUNC,
                  0644);

    if (fd < 0)
        return 1;

    dup2(fd, STDOUT_FILENO);
    dup2(fd, STDERR_FILENO);

    close(fd);

    printf("Normal output\n");
    fprintf(stderr, "Error output\n");

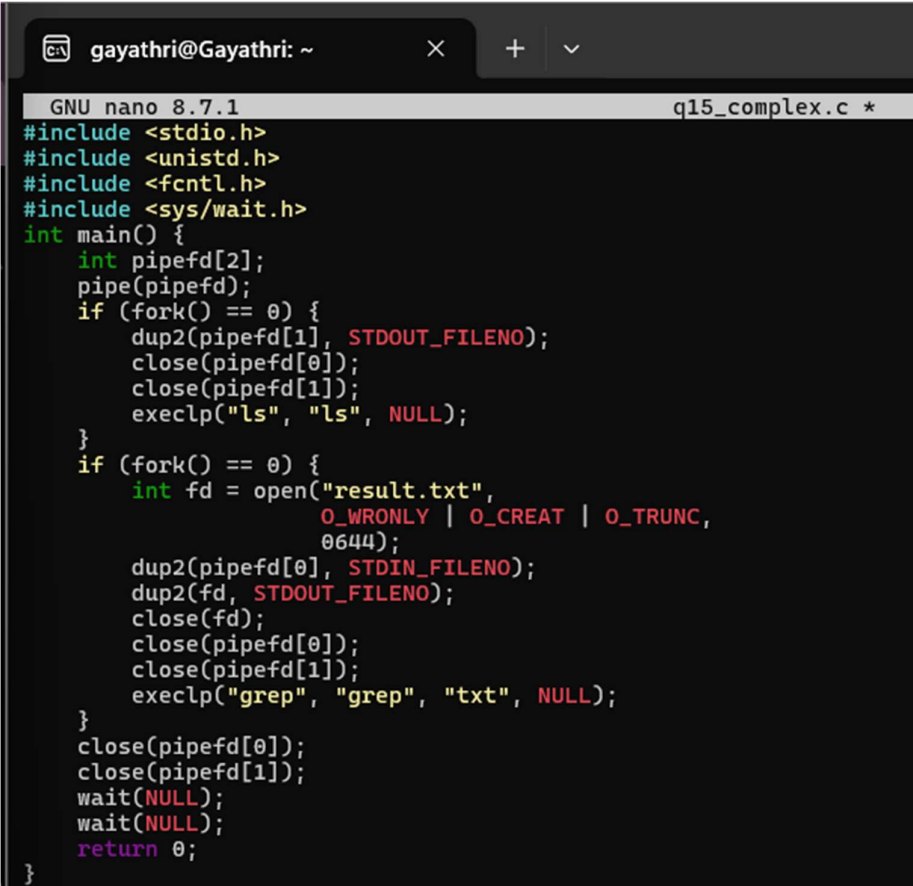
    return 0;
}
```

```

gayathri@Gayathri:~$ nano q15_combined.c
gayathri@Gayathri:~$ gcc q15_combined.c -o q15_combined
gayathri@Gayathri:~$ ./q15_combined
gayathri@Gayathri:~$ cat combined.txt
Error output
Normal output
gayathri@Gayathri:~$

```

15B) Code and Output:



```

GNU nano 8.7.1 q15_complex.c *
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
int main() {
    int pipefd[2];
    pipe(pipefd);
    if (fork() == 0) {
        dup2(pipefd[1], STDOUT_FILENO);
        close(pipefd[0]);
        close(pipefd[1]);
        execlp("ls", "ls", NULL);
    }
    if (fork() == 0) {
        int fd = open("result.txt",
                     O_WRONLY | O_CREAT | O_TRUNC,
                     0644);
        dup2(pipefd[0], STDIN_FILENO);
        dup2(fd, STDOUT_FILENO);
        close(fd);
        close(pipefd[0]);
        close(pipefd[1]);
        execlp("grep", "grep", "txt", NULL);
    }
    close(pipefd[0]);
    close(pipefd[1]);
    wait(NULL);
    wait(NULL);
    return 0;
}

```

```

gayathri@Gayathri:~$ nano q15_complex.c
gayathri@Gayathri:~$ gcc q15_complex.c -o q15_complex
gayathri@Gayathri:~$ ^C
gayathri@Gayathri:~$ ./q15_complex
gayathri@Gayathri:~$ cat result.txt
combined.txt
error.txt
file1.txt
input.txt
log.txt
output.txt
result.txt
gayathri@Gayathri:~$

```

16. To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.

To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records.

16A) Code and Output:

```
gayathri@Gayathri: ~  
GNU nano 8.7.1 q16_background.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <stdlib.h>  
int main() {  
    pid_t pid = fork();  
    if (pid == 0) {  
        sleep(5);  
        printf("Background job completed\n");  
        exit(0);  
    }  
    printf("Background job started with PID: %d\n", pid);  
    printf("Shell prompt returned immediately\n");  
    return 0;  
}  
  
gayathri@Gayathri:~$ nano q16_background.c  
gayathri@Gayathri:~$ gcc q16_background.c -o q16_background  
gayathri@Gayathri:~$ ./q16_background  
Background job started with PID: 1801  
Shell prompt returned immediately  
gayathri@Gayathri:~$ Background job completed
```

16B) Code and Output:

```
gayathri@Gayathri: ~  
GNU nano 8.7.1 q16_jobs.c *  
#include <stdio.h>  
#include <unistd.h>  
  
struct Job {  
    int id;  
    pid_t pid;  
    char command[50];  
};  
  
int main() {  
    struct Job jobs[3];  
  
    jobs[0].id = 1;  
    jobs[0].pid = 1234;  
    snprintf(jobs[0].command, 50, "sleep 10");  
  
    jobs[1].id = 2;  
    jobs[1].pid = 1235;  
    snprintf(jobs[1].command, 50, "sleep 20");  
  
    printf("Job Table:\n");  
    printf("[%d] PID=%d %s\n",  
           jobs[0].id, jobs[0].pid, jobs[0].command);  
  
    printf("[%d] PID=%d %s\n",  
           jobs[1].id, jobs[1].pid, jobs[1].command);  
  
    return 0;  
}
```

```

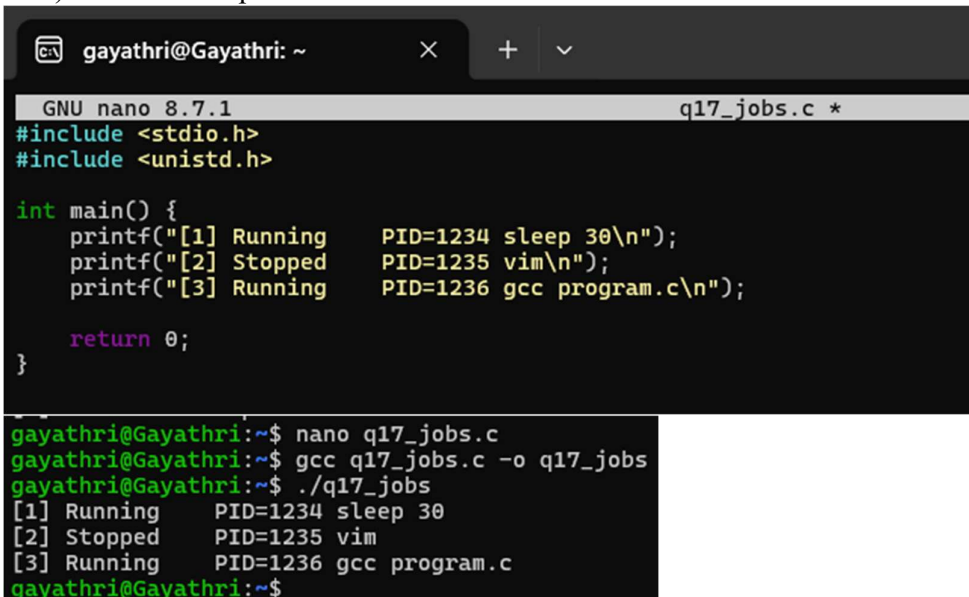
gayathri@Gayathri:~$ nano q16_jobs.c
gayathri@Gayathri:~$ gcc q16_jobs.c -o q16_jobs
gayathri@Gayathri:~$ ./q16_jobs
Job Table:
[1] PID=1234 sleep 10
[2] PID=1235 sleep 20
gayathri@Gayathri:~$ |

```

17.To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.

To design how to Identify Target Job, Transfer Terminal Control, Resume Processes, Wait for Completion, Update Job State, Test Foreground Switching.

17A) Code and Output:



```

GNU nano 8.7.1 q17_jobs.c *
#include <stdio.h>
#include <unistd.h>

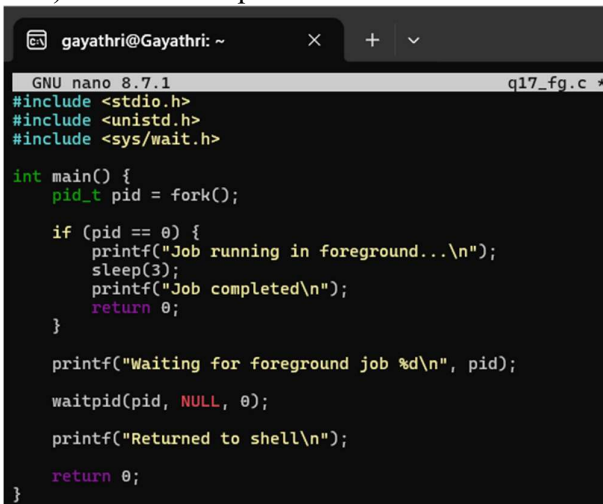
int main() {
    printf("[1] Running    PID=1234 sleep 30\n");
    printf("[2] Stopped    PID=1235 vim\n");
    printf("[3] Running    PID=1236 gcc program.c\n");

    return 0;
}

gayathri@Gayathri:~$ nano q17_jobs.c
gayathri@Gayathri:~$ gcc q17_jobs.c -o q17_jobs
gayathri@Gayathri:~$ ./q17_jobs
[1] Running    PID=1234 sleep 30
[2] Stopped    PID=1235 vim
[3] Running    PID=1236 gcc program.c
gayathri@Gayathri:~$

```

17B) Code and Output:



```

GNU nano 8.7.1 q17_fg.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Job running in foreground...\n");
        sleep(3);
        printf("Job completed\n");
        return 0;
    }

    printf("Waiting for foreground job %d\n", pid);
    waitpid(pid, NULL, 0);
    printf("Returned to shell\n");

    return 0;
}

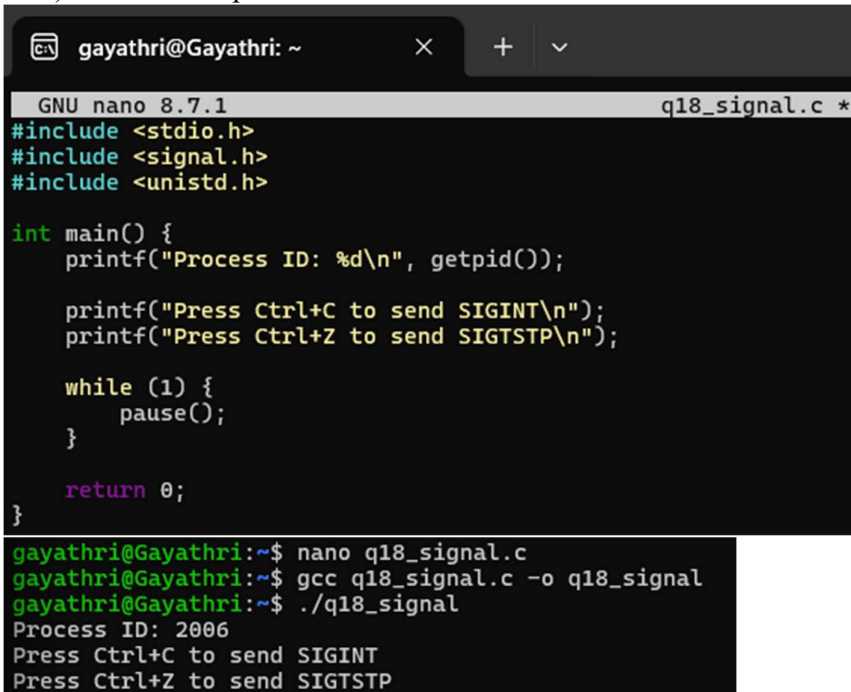
```

```
gayathri@Gayathri:~$ nano q17_fg.c
gayathri@Gayathri:~$ gcc q17_fg.c -o q17_fg
gayathri@Gayathri:~$ ./q17_fg
Waiting for foreground job 1915
Job running in foreground...
Job completed
Returned to shell
gayathri@Gayathri:~$
```

18.To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior.

To implement how to resume Stopped Jobs, Send Continue Signals, Update Job Status, Maintain Job Table, Verify Execution, Test Background Recovery.

18A) Code and output:



```
GNU nano 8.7.1 q18_signal.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int main() {
    printf("Process ID: %d\n", getpid());

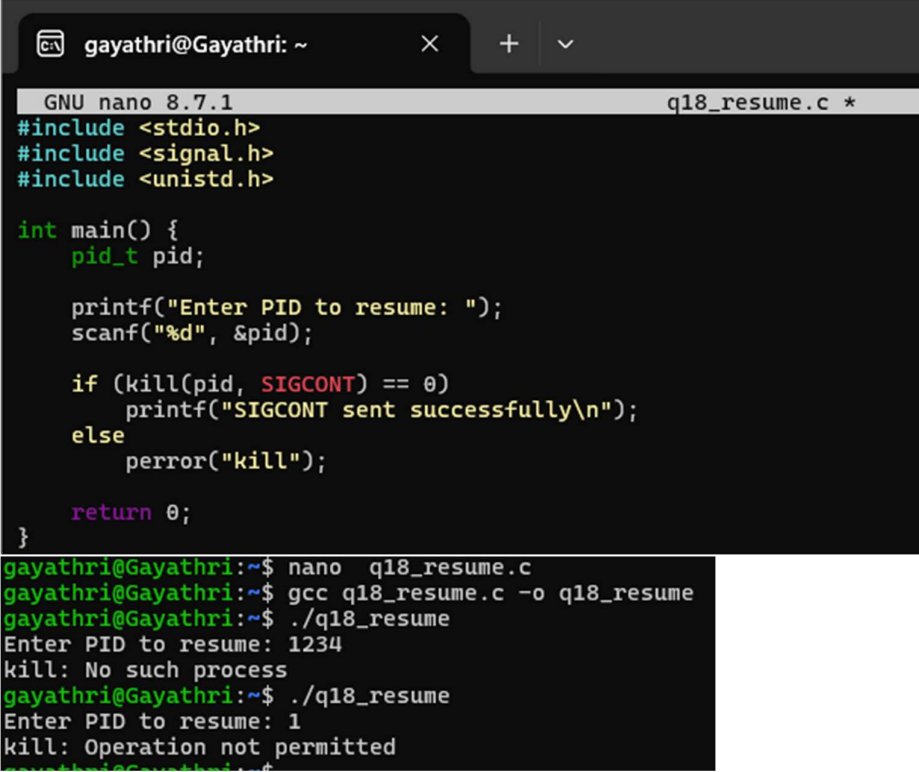
    printf("Press Ctrl+C to send SIGINT\n");
    printf("Press Ctrl+Z to send SIGTSTP\n");

    while (1) {
        pause();
    }

    return 0;
}

gayathri@Gayathri:~$ nano q18_signal.c
gayathri@Gayathri:~$ gcc q18_signal.c -o q18_signal
gayathri@Gayathri:~$ ./q18_signal
Process ID: 2006
Press Ctrl+C to send SIGINT
Press Ctrl+Z to send SIGTSTP
```

18B) Code and Output:



```
GNU nano 8.7.1 q18_resume.c *
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

int main() {
    pid_t pid;

    printf("Enter PID to resume: ");
    scanf("%d", &pid);

    if (kill(pid, SIGCONT) == 0)
        printf("SIGCONT sent successfully\n");
    else
        perror("kill");

    return 0;
}

gayathri@Gayathri:~$ nano q18_resume.c
gayathri@Gayathri:~$ gcc q18_resume.c -o q18_resume
gayathri@Gayathri:~$ ./q18_resume
Enter PID to resume: 1234
kill: No such process
gayathri@Gayathri:~$ ./q18_resume
Enter PID to resume: 1
kill: Operation not permitted
gayathri@Gayathri:~$
```

19.To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.

19A) Code and Output:

```
gayathri@Gayathri: ~  
GNU nano 8.7.1 q19_handler.c *  
#include <stdio.h>  
#include <signal.h>  
#include <unistd.h>  
  
void handler(int signal) {  
    printf("\nSignal received: %d\n", signal);  
}  
  
int main() {  
    signal(SIGINT, handler);  
  
    printf("Running... Press Ctrl+C\n");  
  
    while (1)  
        sleep(1);  
  
    return 0;  
}
```

```
kill: operation not permitted  
gayathri@Gayathri:~$ nano q19_handler.c  
gayathri@Gayathri:~$ gcc q19_handler.c -o q19_handler  
gayathri@Gayathri:~$ ./q19_handler  
Running... Press Ctrl+C  
  
Signal received: 2^\Quit  
gayathri@Gayathri:~$ |
```

19B) Code and Output:

```
gayathri@Gayathri: ~  
GNU nano 8.7.1 q19_sigint.c *  
#include <stdio.h>  
#include <signal.h>  
#include <unistd.h>  
#include <sys/wait.h>  
#include <stdlib.h>  
pid_t child;  
void sigint_handler(int sig)  
{  
    printf("\nShell received SIGINT\n");  
    if (child > 0)  
        kill(child, SIGINT);  
}  
int main()  
{  
    signal(SIGINT, sigint_handler);  
    child = fork();  
    if (child < 0)  
    {  
        perror("fork");  
        return 1;  
    }  
    if (child == 0)  
    {  
        signal(SIGINT, SIG_DFL);  
        while (1)  
        {  
            printf("Child running...\n");  
            sleep(1);  
        }  
    }  
    waitpid(child, NULL, 0);  
    printf("Child terminated\n");  
    return 0;  
}
```

```
gayathri@Gayathri: ~  
^C  
gayathri@Gayathri:~$ nano q19_sigint.c  
gayathri@Gayathri:~$ gcc q19_sigint.c -o q19_sigint  
gayathri@Gayathri:~$ nano q19_sigint.c  
gayathri@Gayathri:~$ gcc q19_sigint.c -o q19_sigint  
gayathri@Gayathri:~$ ./q19_sigint  
Child running...  
Child running...  
Child running...  
^C  
Shell received SIGINT  
Child terminated  
gayathri@Gayathri:~$
```

20.To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

20A) Code and Output:

```
gayathri@Gayathri: ~  
GNU nano 8.7.1 q20_sigstsp.c *  
#include <stdio.h>  
#include <signal.h>  
#include <unistd.h>  
  
void stop_handler(int sig) {  
    printf("\nSIGTSTP received\n");  
    printf("Process can be stopped here\n");  
}  
  
int main() {  
    signal(SIGTSTP, stop_handler);  
  
    printf("Process running. Press Ctrl+Z\n");  
  
    while (1)  
        sleep(1);  
  
    return 0;  
}  
  
gayathri@Gayathri:~$ nano q20_sigstsp.c  
gayathri@Gayathri:~$ gcc q20_sigstsp.c -o q20_sigstsp  
gayathri@Gayathri:~$ ./q20_sigstsp  
Process running. Press Ctrl+Z  
^Z  
SIGTSTP received  
Process can be stopped here
```

20B) Code and Output:

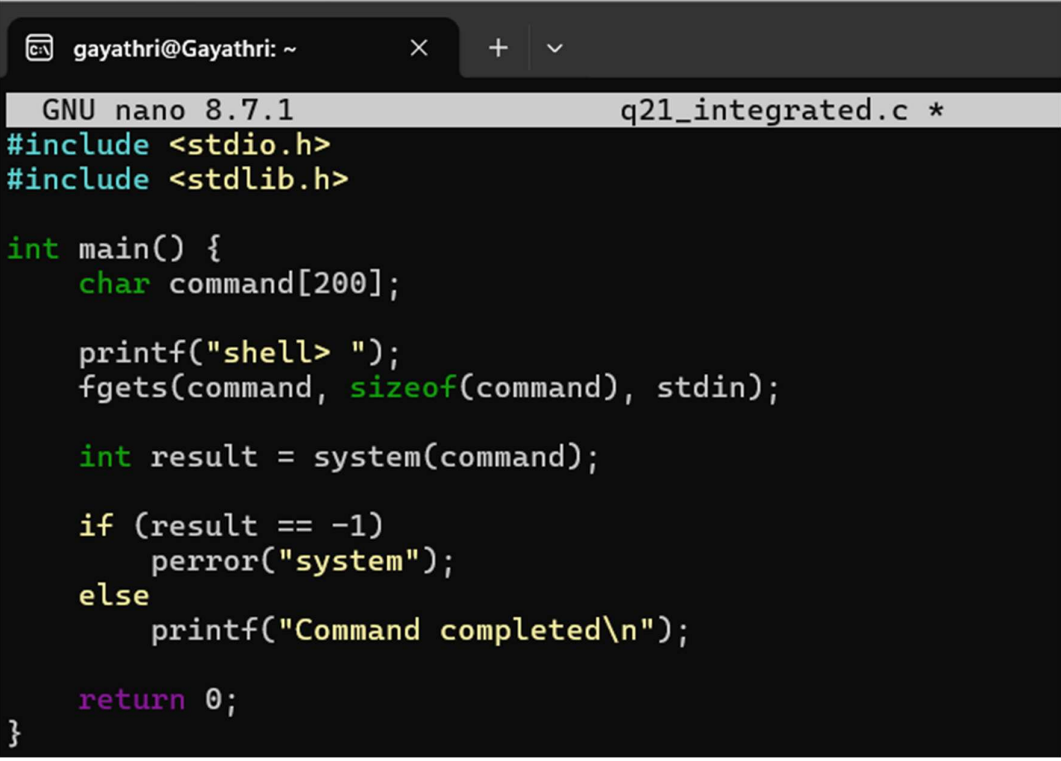
```
gayathri@Gayathri: ~  
GNU nano 8.7.1 q20_processgroup.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main() {  
    pid_t pid = fork();  
  
    if (pid == 0) {  
        setpgid(0, 0);  
  
        printf("Child PID: %d\n", getpid());  
        printf("Process Group ID: %d\n", getpgrp());  
  
        sleep(5);  
  
        return 0;  
    }  
  
    wait(NULL);  
  
    printf("Parent finished\n");  
  
    return 0;  
}
```

```
gayathri@Gayathri:~$ nano q20_processgroup.c  
gayathri@Gayathri:~$ gcc q20_processgroup.c -o q20_processgroup  
gayathri@Gayathri:~$ ./q20_processgroup  
Child PID: 2362  
Process Group ID: 2362  
Parent finished
```

21.To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

21A) Code and Output:



```
gayathri@Gayathri: ~  
GNU nano 8.7.1 q21_integrated.c *  
#include <stdio.h>  
#include <stdlib.h>  
  
int main() {  
    char command[200];  
  
    printf("shell> ");  
    fgets(command, sizeof(command), stdin);  
  
    int result = system(command);  
  
    if (result == -1)  
        perror("system");  
    else  
        printf("Command completed\n");  
  
    return 0;  
}
```

```

gayathri@Gayathri:~$ nano q21_integrated.c
gayathri@Gayathri:~$ gcc q21_integrated.c -o q21_integrated
gayathri@Gayathri:~$ ./q21_integrated
shell> ls
Address_Variable      nano.2179.save      q17_fg.c
Address_Variable.c    nano.2238.save      q17_jobs
Childwait             null_pointer.c      q17_jobs.c
Childwait.c           null_pointer.c      q18_resume
CopytoOther           orphan.c             q18_resume.c
CopytoOther.c         orphan.c             q18_signal
Hello                 output.txt           q18_signal.c
Hello.c               pagetable.c          q19_handler
Linux                 pagetable.c          q19_handler.c
Linux.c               pcec.c              q19_sigint
MultipleChild          pcec.c              q19_sigint.c
MultipleChild.c        pipe.c              q20_processgroup
'OS Project'          pipe.c              q20_processgroup.c
OSLAB                 practical3           q20_sigstsp
PCEC                  practical3.c         q20_sigstsp.c
ParentChild           practical4           q21_integrated
ParentChild.c          practical4.c         q21_integrated.c
Pcpid                 practical5           q5_double
Pcpid.c               practical5.c         q5_double.c
Pcrun                 practical6           q6_escape
Pcrun.c               practical6.c         q6_escape.c
Signal                practical7           q6_exec
Signal.c              practical7.c         q6_exec.c
address.translation    practical6           q7_path
address.translation.c  practical6           q7_path.c
childfork             practical8a          q7_waitpid
childfork.c           practical8a.c        q7_waitpid.c
client                priority.c           q8_builtin
client.c              priority.c           q8_builtin.c
combined.txt          process-lab          q8_variable
command_history.c     processid            q8_variable.c
copyfile              processid.c          q9_cd
copyfile.c            program1             q9_cd.c
cow                   program1.c          q9_dispatch
cow.c                 program2             q9_dispatch.c
error.txt             program2.c           result.txt
exclp                 prrogram2            round_robin
exclp.c              q10_export           round_robin.c
fcfs                  q10_export.c        s3
fcfs.c               q10_shell            s3.c
fifo_write            q10_shell.c         s4
fifo_write.c          q11_history           s4.c
fifo_read             q11_history.c        s5
fifo_read.c           q11_pipeline         s5.c
fifo_write            q11_pipeline.c      server
fifo_write.c          q12_multipipe        server.c
fifo_write.c.save     q12_multipipe.c     sigchild
fifo_writel           q12_pipe             sigchild.c
fifo_writel.c         q12_pipe.c           sigpipe
file1.tx              q13_input            sigpipe.c
file1.txt             q13_input.c          sigseg
forge                 q13_output           sigseg.c
forge.c               q13_output.c         sigterm
fork                  q14_append           sigterm.c
fork.c                q14_append.c        sjf_nonpreemptive
fork.c=="             q14_stderr           sjf_nonpreemptive.c
github-c-files        q14_stderr.c         skill3
hello.c               q15_combined         skill3.c
input.txt             q15_combined.c       skill6
linux.c               q15_complex          skill6.c
log.txt               q15_complex.c        socket.c
malloc                q16_background       task_management
malloc.c              q16_background.c     write.c
memory                q16_jobs             zombie
memory.c              q16_jobs.c           zombie.c
myfifo                q17_fg
Command completed
gayathri@Gayathri:~$ |

```

21B) Code and Output:

```

GNU nano 8.7.1 q21_error.c *
#include <stdio.h>
#include <stdlib.h>

int main() {
    int a, b;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    if (b == 0) {
        fprintf(stderr, "Error: Division by zero\n");
        return 1;
    }

    printf("Result = %d\n", a / b);

    return 0;
}

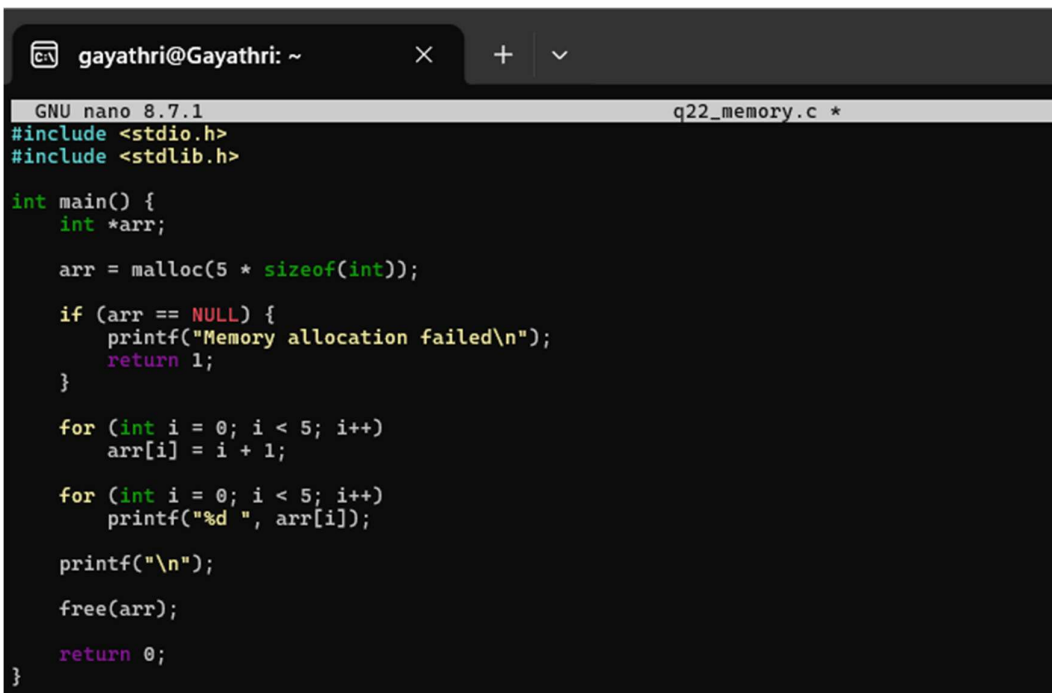
```

```
gayathri@Gayathri:~$ nano q21_error.c
gayathri@Gayathri:~$ gcc q21_error.c -o q21_error
gayathri@Gayathri:~$ ./q21_error
Enter two numbers: 10 0
Error: Division by zero
gayathri@Gayathri:~$ |
```

22.To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

22A) Code and Output:



```
GNU nano 8.7.1 q22_memory.c *
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr;

    arr = malloc(5 * sizeof(int));

    if (arr == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
        arr[i] = i + 1;

    for (int i = 0; i < 5; i++)
        printf("%d ", arr[i]);

    printf("\n");

    free(arr);

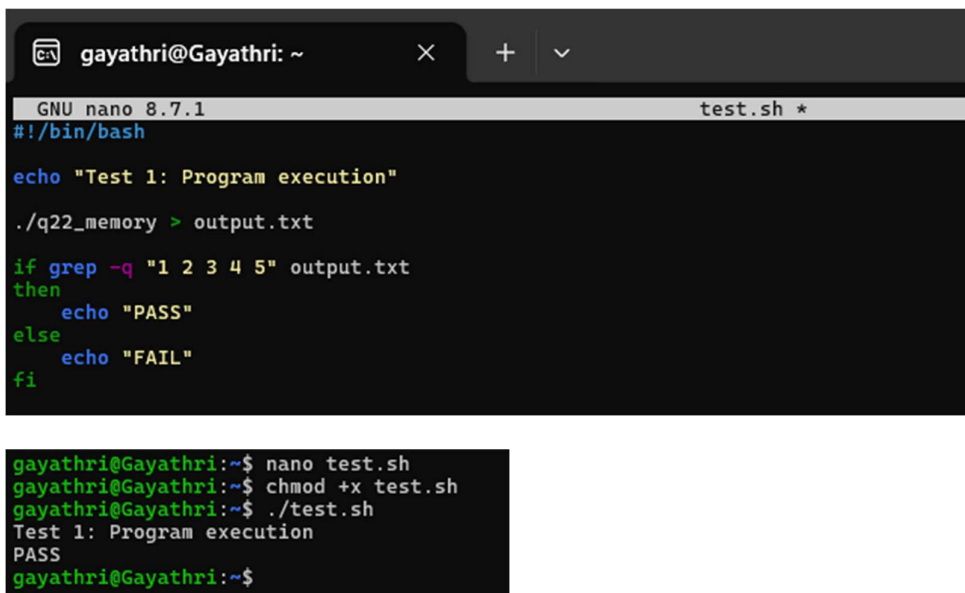
    return 0;
}
```

```

gayathri@Gayathri:~$ nano q22_memory.c
gayathri@Gayathri:~$ gcc -g q22_memory.c -o q22_memory
gayathri@Gayathri:~$ ./q22_memory
1 2 3 4 5
gayathri@Gayathri:~$ valgrind --leak-check=full ./q22_memory
==2481== Memcheck, a memory error detector
==2481== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2481== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2481== Command: ./q22_memory
==2481==
1 2 3 4 5
==2481==
==2481== HEAP SUMMARY:
==2481==   in use at exit: 0 bytes in 0 blocks
==2481== total heap usage: 2 allocs, 2 frees, 1,044 bytes allocated
==2481==
==2481== All heap blocks were freed -- no leaks are possible
==2481==
==2481== For lists of detected and suppressed errors, rerun with: -s
==2481== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
gayathri@Gayathri:~$ |

```

22B) Code and Output:



```

GNU nano 8.7.1 test.sh *
#!/bin/bash

echo "Test 1: Program execution"

./q22_memory > output.txt

if grep -q "1 2 3 4 5" output.txt
then
    echo "PASS"
else
    echo "FAIL"
fi

gayathri@Gayathri:~$ nano test.sh
gayathri@Gayathri:~$ chmod +x test.sh
gayathri@Gayathri:~$ ./test.sh
Test 1: Program execution
PASS
gayathri@Gayathri:~$

```

23.To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance.

To Document Architecture, Create Flow Diagrams, Explain Execution Paths, Describe Syscalls, Record Design Decisions, Review Documentation.

23) Code and Output:

```
gayathri@Gayathri: ~  
GNU nano 8.7.1 q23_pipeline.c *  
#include <stdio.h>  
#include <stdlib.h>  
  
int main() {  
    printf("Executing large pipeline...\n");  
    system("ls | sort | uniq | wc -l");  
    printf("Pipeline completed successfully\n");  
    return 0;  
}  
  
PASS  
gayathri@Gayathri:~$ nano q23_pipeline.c  
gayathri@Gayathri:~$ gcc q23_pipeline.c -o q23_pipeline  
gayathri@Gayathri:~$ ./q23_pipeline  
Executing large pipeline...  
204  
Pipeline completed successfully  
gayathri@Gayathri:~$ time ./q23_pipeline  
Executing large pipeline...  
204  
Pipeline completed successfully  
  
real    0m0.011s  
user    0m0.016s  
sys     0m0.007s  
gayathri@Gayathri:~$
```

24.To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables.

To Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review.

24)Code and Output:

```
gayathri@Gayathri: ~  
GNU nano 8.7.1 Makefile *  
CC = gcc  
CFLAGS = -Wall -Wextra  
  
TARGET = ossp_shell  
  
all: $(TARGET)  
  
$(TARGET): main.c  
    $(CC) $(CFLAGS) main.c -o $(TARGET)  
  
run: $(TARGET)  
    ./$(TARGET)  
  
clean:  
    rm -f $(TARGET)  
  
test: $(TARGET)  
    ./$(TARGET)  
  
.PHONY: all run clean test
```

